

REPORTE FINAL DE FUNDAMENTOS MATEMATICOS

1. PRODUCTO FINAL

Como producto final se entrega un reporte tecnico que integra investigacion, analisis matematico, reglas logicas, detector de anomalias, cifrado multiplicativo, resultados y conclusiones del simulador.

El sistema simula una red institucional durante 60 segundos. En cada ciclo se generan datos de trafico digital y se evalua si el comportamiento es normal o representa riesgo de ciberseguridad.

2. FORMULA GENERAL

En fundamentos matematicos se usan formulas generales de logica y cifrado.

Regla logica general:

text

Si condicion A se cumple, entonces activar respuesta B

Cifrado modular general:

text

$$c = (m * k) \bmod n$$

Donde m es el valor original, k es la clave, n es el modulo y c es el valor cifrado. Esta formula se usa para explicar el cifrado multiplicativo de letras y numeros.

3. INVESTIGACION SOBRE ATAQUES DE TRAFICO DIGITAL Y CIBERSEGURIDAD

Un ataque de trafico digital ocurre cuando una red recibe actividad anormal que puede afectar disponibilidad, confidencialidad o integridad. En el simulador se representan varios tipos:

Ataque	Representacion en el programa	Riesgo
trafico masivo	aumento de paquetes	saturacion del servidor
fuerza bruta	aumento de intentos de login	acceso no autorizado
acceso no autorizado	usuario sospechoso e IP externa	ingreso indebido
copia de expediente	salida de datos elevada	fuga de informacion
modificacion de evidencia	cambios de archivos	alteracion de registros
eliminacion de archivo	cambios y salida de datos	perdida de informacion
extraccion de informacion	paquetes, datos y login elevados	exfiltracion

La ciberseguridad busca detectar estos patrones, activar protecciones y mantener la continuidad del servicio.

4. DATOS GENERADOS POR EL SIMULADOR

Cada ciclo genera un paquete con datos como:

Variable	Significado
paquetes	volumen de trafico recibido
intentos_login	intentos de acceso
cambios_archivos	modificaciones sobre expedientes
salida_datos	datos que salen del sistema
usuario	agente o usuario sospechoso
ip	direccion de origen
cpu	carga del servidor
temperatura	consecuencia fisica de la carga
estado	clasificacion del sistema
cifrado	nivel de proteccion de la ruta

Los datos normales se generan en m02_trafico_normal.py y los ataques se inyectan en m03_atacante.py.

5. ANALISIS MATEMATICO DE LOS DATOS

El sistema combina las variables principales en un indice llamado flujo digital:

text

$$\text{Flujo} = 0.45P + 0.25L + 0.15A + 0.15D$$

Donde:

Simbolo	Variable
P	paquetes
L	intentos de login
A	cambios de archivos
D	salida de datos

Los pesos se eligieron porque los paquetes son el indicador mas importante de trafico, los intentos de login representan acceso, los cambios de archivos representan alteracion y la salida de datos representa posible fuga de informacion.

Ejemplo de interpretacion:

text

Si P aumenta mucho, el flujo sube.

Si L aumenta junto con P, el sistema puede detectar ataque coordinado.

Si D aumenta, se interpreta como riesgo de exfiltracion.

6. INTERPRETACION DE GRAFICAS Y TASAS DE CAMBIO

El programa genera graficas para observar el comportamiento en el tiempo:

Grafica	Interpretacion
01 paquetes vs tiempo	muestra picos de trafico y caida final en los ultimos 5 ciclos
02 intentos login	permite observar intentos normales y fuerza bruta

03 temperatura	compara temperatura antes y despues de enfriamiento
04 estado del sistema	muestra cambios entre normal, sospechoso, alerta y critico
05 flujo digital	muestra cuando se superan umbrales de riesgo
06 derivadas	analiza velocidad y aceleracion del cambio
07 analisis oscilatorio	compara comportamiento real contra una onda regular

La tasa de cambio se calcula como:

text

Tasa = paquetes actuales - paquetes anteriores

Tambien se usa derivada discreta:

text

$f'(t) = (f(t) - f(t-1)) / 1$

Esto permite detectar subidas rapidas aunque el valor total todavia no parezca critico.

7. REGLAS LOGICAS DE DETECCION

El detector usa reglas condicionales. Algunas reglas principales son:

| Regla | Resultado |

|---|---|

| usuario no autorizado | estado ALERTA |

| paquetes altos + login alto | estado ALERTA y cifrado |

| cambios de archivos altos | alerta por manipulacion |

| salida de datos elevada | cifrado por posible fuga |

| paquetes criticos o flujo critico | estado CRITICO y cifrado reforzado |

| oscilacion critica | estado CRITICO |

| intentos login criticos | login remoto cerrado |

Estas reglas representan proposiciones logicas del tipo:

text

Si condicion A y condicion B se cumplen, entonces activar proteccion C.

Ejemplo:

text

Si paquetes > U_PAQUETES_ALERTA y login > U_LOGIN_ALERTA,
entonces estado = ALERTA.

8. IMPLEMENTACION COMPUTACIONAL DEL DETECTOR

La funcion central es evaluar_estado en m04_logica_matematica.py. Esta funcion recibe el paquete, el flujo digital, el cifrado actual y el indice oscilatorio. Devuelve:

text

estado, alertas, protecciones, cifrado

El ciclo principal usa esos valores para actualizar:

```
| Valor | Uso |  
|---|---|  
| estado_actual["estado"] | estado visible del sistema |  
| estado_actual["estado_arrastrado"] | ultimo estado importante no normal |  
| estado_actual["cifrado"] | nivel de cifrado |  
| alertas | mensajes de riesgo |  
| protecciones | acciones defensivas |
```

En los ultimos 5 ciclos la probabilidad de ataque se vuelve 0.00, para mostrar estabilizacion y caida de ataques en las graficas. Al final se pregunta si se quieren desactivar protecciones. Si la respuesta es afirmativa, el estado final pasa a NORMAL; si no, se conserva el estado arrastrado.

9. SISTEMA DE CIFRADO MULTIPLICATIVO

El proyecto implementa cifrado Cesar y cifrado multiplicativo. El cifrado reforzado combina ambos.

9.1 Cifrado Cesar

```
text  
posicion_cifrada = (posicion + desplazamiento) mod 26
```

En el programa el desplazamiento es 3.

9.2 Cifrado multiplicativo

Para letras:

```
text  
posicion_cifrada = (posicion * clave) mod 26
```

La clave usada para letras es 5, que funciona porque tiene inverso modular en modulo 26.

Para numeros se usa una forma afin:

```
text  
digito_cifrado = (digito * 7 + 3) mod 10
```

El descifrado requiere inverso modular:

```
text  
clave * inverso mod modulo = 1
```

9.3 Ruta cifrada

La ruta interna se cifra sin mostrar `http://servidor/`. Ejemplo:

text

Cifrado: `fhqwudo_qruwh/fl-2026-488108/gdwrw_12.orj`

Servidor: `http://servidor/central_norte/ci-2026-488108/datos_12.log`

Esto permite presentar una direccion protegida y, al mismo tiempo, mostrar que el servidor puede descifrar la ruta original.

10. RESULTADOS OBTENIDOS

Ultima ejecucion registrada:

Resultado Valor
--- ---
ciclos ejecutados 60
incidentes detectados 27
paquetes totales 34513
intentos login totales 134
cambios totales 500
salidas totales 2291 MB
ciclos NORMAL 34
ciclos SOSPECHOSO 2
ciclos ALERTA 15
ciclos CRITICO 9
ultimos 5 ciclos con ataque 0
cifrado final del historial REFORZADO
flujo maximo 1413.6
indice oscilatorio maximo 3069.67

El resultado muestra que el detector identifica periodos normales, alertas y estados criticos. Tambien muestra una caida final de ataques, lo que ayuda a interpretar que el sistema se estabilizo aunque el cifrado permanezca reforzado.

11. CONCLUSIONES Y POSIBLES MEJORAS

El proyecto demuestra que los fundamentos matematicos permiten construir reglas de deteccion, medir cambios, comparar umbrales y proteger rutas mediante cifrado. Las proposiciones logicas permiten decidir estados; las operaciones modulares permiten cifrar; y las tasas de cambio permiten detectar anomalias.

Posibles mejoras:

Mejora Beneficio
--- ---
usar datos reales de red mejorar realismo
ajustar pesos del flujo hacer el detector mas preciso
guardar varias simulaciones comparar resultados
agregar probabilidad configurable probar distintos escenarios
mejorar cifrado usar claves dinamicas

| agregar interfaz grafica | facilitar uso del sistema |